Test Paper 2 (Image 2)

Q1. How do software engineering principles help develop software products cost-effectively and timely? [02]

- Software engineering provides **structured processes** (phases like requirements → design → coding → testing) that make time and cost estimation possible
- **Early error detection** — catching bugs in design phase is far cheaper than fixing them post-deployment
- **Modularity** allows parallel development, saving time
- **Reusability** of components reduces redundant work and cost
- **Documentation and standards** prevent miscommunication and rework

Q2. Why do characteristics of requirements play a significant role in selecting a life cycle model? [02]

- **Stable, clear requirements** → Waterfall model (linear, phase-by-phase)
- **Vague or evolving requirements** → Agile or Spiral (iterative, flexible)
- **High-risk requirements** → Spiral model (built-in risk analysis each cycle)
- **Unclear user needs** → Prototyping model (build to clarify)
- Wrong model + wrong requirements = primary cause of **project failure**

Q3. Categories of SRS users and their expectations [02]

User	Expectation
Client/Customer	Correct reflection of business needs in plain language
Developers	Precise, technically detailed, unambiguous specs
Testers	Every requirement must be verifiable and testable
Project Managers	Consistent, traceable for planning and scheduling

Q4. RAD Model — advantages, disadvantages, when to use [02]

What it is:

- Iterative model focused on **speed** using reusable components, parallel teams, and constant user feedback

Advantages:

- Faster delivery (60–90 days per module)
- High user satisfaction due to continuous involvement
- Flexible to changing requirements

- Reusable components reduce effort

Disadvantages:

- Requires highly skilled developers
- Needs **active, available client** — if absent, model breaks down
- Not ideal for high-performance or technically complex systems
- Large systems hard to modularize effectively

When to use:

- Requirements are reasonably well-known
- Project can be split into independent modules
- Client can commit to ongoing involvement
- Best for: business apps, dashboards, internal tools

Test Paper 1 (Image 1)

Q1. Distinguish between generic and customized software [02]

Aspect	Generic Software	Customized Software
Target	Broad market	Specific client
Specification owner	Developer	Client
Cost	Cheaper, off-the-shelf	Expensive, built from scratch
Fit	May require compromise	Perfect fit for needs
Examples	MS Word, Photoshop	Hospital management system

Q2. What does "software is invisible" mean? [02]

- Software has **no physical form** — cannot be seen or touched unlike a bridge or circuit board
- Progress is **hard to measure** — looks 90% done but may hide critical bugs
- **Defects don't show visually** — they hide until triggered by a specific input
- Communication with clients is harder — nothing concrete to point to
- This is why we use **UML diagrams, flowcharts, architectural models** — to make the invisible visible

---

Q3. Characteristics of a good SRS [02]

- **Correct** — accurately reflects what the system must do
- **Unambiguous** — one and only one interpretation per requirement
- **Complete** — covers all functional, non-functional, interface, and constraint requirements
- **Consistent** — no two requirements contradict each other
- **Verifiable** — every requirement can be tested

- **Modifiable** — changes can be made cleanly without ripple effects
- **Traceable** — each requirement links back to its origin and forward to implementation
- **Ranked** — requirements ordered by importance or stability

---

Q4. Throwaway prototype — how to respond to client wanting to deliver it as final system [04]

Step 1 — Explain what a throwaway prototype is:

- Built quickly to **clarify requirements only**
- Intentional shortcuts taken — no production-quality standards
- Meant to be **discarded**, not deployed

Step 2 — Explain the specific risks:

- **Maintainability** — poorly structured, undocumented, fragile code
- **Security** — no proper authentication or data validation
- **Performance** — never tested under real load
- **Reliability** — edge cases unhandled
- **Scalability** — will fail as users/data grow

Step 3 — Propose a constructive path forward:

- Use the prototype as a **validated blueprint**
- Build a proper system based on its design insights
- Show client that cost now prevents **far greater costs later**

Step 4 — Document your concerns formally:

- If client still insists, put your objections **in writing**
- Protects you legally and professionally
- Often makes clients reconsider when they see it formally stated

> 💡 **Key principle:** Your duty as a software engineer is not just to deliver what the client asks, but to act in their **genuine best interest** — and deploying a fragile prototype does not serve that interest.

I'll answer all the questions from both test papers systematically, matching the depth to the marks allocated.

Test Paper 2 (Image 2) — Software Engineering, Feb 2025

Q1. How do software engineering principles help develop software products cost-effectively and timely? [02]

Software engineering principles bring **discipline and structure** to what would otherwise be a chaotic development process. Without these principles, developers would write code ad hoc, leading to missed deadlines, budget overruns, and fragile systems.

The key way these principles help is through **systematic planning and process models** — by defining clear phases (requirements → design → coding → testing → maintenance), teams can estimate time and cost at each stage, catch errors early (when they're cheap to fix), and avoid rework. For example, fixing a requirements error during the design phase costs far less than fixing it after deployment.

Principles like **modularity** (breaking a system into manageable components) and **abstraction** (hiding complexity) allow multiple developers to work in parallel, saving time. **Reusability** — designing components that can be used across projects — directly reduces cost. Together, these principles turn software development from an art into an engineering discipline with predictable outcomes.

Q2. Why do characteristics of requirements play a significant role in selecting a life cycle model? [02]

The life cycle model you choose is essentially your **game plan for the entire project**, so it must be matched to the nature of your requirements. Think of it like choosing between building a house (fixed blueprint) versus designing a custom suit (iterative fittings) — the right approach depends on how well-defined your needs are.

If requirements are **clear, stable, and well-understood upfront**, a linear model like the **Waterfall model** works well because you can move confidently from one phase to the next. But if requirements are **vague, evolving, or poorly understood**, an iterative model like **Agile or Spiral** is more appropriate because it accommodates change through repeated cycles.

Other characteristics matter too: if requirements carry **high risk** (unclear technology, new domain), the Spiral model's risk-analysis loops are ideal. If requirements demand a **quick prototype** to clarify user needs, RAD or prototyping models suit better. In short, forcing the wrong model onto a set of requirements is a primary cause of project failure.

Q3. Who are the different categories of users of the SRS document? What are their expectations? [02]

The Software Requirements Specification (SRS) document serves as a **contract between all stakeholders**, so it naturally has multiple audiences with different needs.

Clients and customers use the SRS to verify that the development team has correctly understood what they want. Their expectation is that the document reflects their business needs in plain, unambiguous language — they are less concerned with technical details and more with functional correctness.

Developers and designers use it as a blueprint to guide system architecture and coding. They expect the SRS to be precise, complete, and technically specific enough to make design decisions without constantly going back to the client.

Testers and QA teams rely on the SRS to derive test cases. Their expectation is that every requirement be **verifiable** — i.e., it should be possible to write a test that confirms whether the requirement has been met.

Project managers use it for planning, scheduling, and resource allocation. They expect it to be consistent and traceable so they can track progress against requirements.

Q4. Explain the RAD Model — its advantages, disadvantages, and when to use it. [02]

The **Rapid Application Development (RAD) model** is an iterative development approach that prioritizes speed by using **reusable components, parallel development teams, and heavy user involvement** throughout the process. Instead of spending months on upfront planning, RAD gets working prototypes in front of users quickly and refines them through feedback loops.

Its **advantages** include significantly faster delivery (typically 60–90 days per module), higher user satisfaction because users are involved throughout, and flexibility to accommodate changing requirements mid-development. Reusing pre-built components also reduces redundant effort.

However, its **disadvantages** are notable: it requires highly skilled developers and active client participation (if the client is unavailable, the model breaks down). It is also not suitable for systems requiring very high performance or where technical risks are significant, since speed sometimes compromises architectural rigor. Large, complex systems can be difficult to modularize effectively for RAD.

When to use RAD: It is best suited for projects with a **2–3 month timeline per module**, where requirements are reasonably well-known, the client can commit to ongoing involvement, and the system can be broken into independent modules. Business applications, dashboards, and internal tools are classic RAD candidates.

Test Paper 1 (Image 1) — Second Test

Q1. Distinguish between generic and customized software products. [02]

Generic software (also called off-the-shelf software) is developed for a **broad market** without a specific customer in mind. The developer controls the specification and decides what features to include. Examples include Microsoft Word, Adobe Photoshop, or antivirus software — products that millions of users buy and use as-is.

Customized (bespoke) software, on the other hand, is developed for a **specific client** with a specific need. Here, the client controls the specification and the software is built precisely to their requirements. Examples include a hospital's patient management system or a bank's internal transaction processing software.

The key distinction lies in **who owns the specification**: with generic software, the developer does; with customized software, the client does. Customized software is typically more expensive and time-consuming but fits the client's needs perfectly, while generic software is cheaper and immediately available but may require compromises.

Q2. What do we mean by saying that software is invisible? [02]

When we say software is **invisible**, we mean that unlike physical engineering products — a bridge, a circuit board, a building — software has **no tangible, physical form** that can be seen or touched. You cannot look at a running program and observe its structure the way an architect can look at a building's blueprints manifested in steel and concrete.

This invisibility creates real engineering challenges. Progress is hard to measure — a project can appear 90% complete while hiding critical unresolved problems deep in the code. Communication between developers and clients is difficult because there's nothing concrete to point to and say "this part is wrong." It also makes quality control harder; defects don't crack or rust visibly — they lurk silently until triggered by the right input.

This is why software engineering developed tools like **flowcharts, UML diagrams, and architectural models** — they are attempts to make the invisible visible, giving teams a shared representation of something that fundamentally has no physical form.

Q3. List the characteristics of a good Software Requirements Specification (SRS). [02]

A good SRS must be **correct** — every stated requirement must accurately reflect what the system needs to do, with no errors or contradictions. It must be **unambiguous**, meaning each requirement has exactly one possible interpretation, leaving no room for guesswork.

It must be **complete**, covering all significant requirements — functional, non-functional, interface, and constraint requirements — with no gaps. It should be **consistent**, meaning no two requirements contradict each other. It must be **verifiable**, so each requirement can be tested or checked against the delivered system.

It should also be **modifiable** (well-structured enough that changes can be made cleanly), **traceable** (each requirement can be traced to its origin and forward to its implementation), and **ranked by importance or stability**, so developers know which requirements are core versus optional. Together, these qualities make the SRS a reliable foundation for the entire project.

Q4. You developed a throwaway prototype and the client wants to deliver it as the final system. How do you respond? [04]

This is a genuinely challenging professional and ethical situation, and your response needs to be **firm but diplomatic** — protecting the client's long-term interests even when they may not immediately see the risk.

First, you should clearly explain what a throwaway prototype is. A throwaway (or exploratory) prototype is built quickly with the sole purpose of clarifying requirements and exploring design options. It is intentionally built **without production-quality standards** — shortcuts are taken in error handling, security, scalability, documentation, and code structure. It is meant to be discarded once it has served its exploratory purpose, not to be the foundation of a live system.

Second, you should articulate the specific risks of delivering it as-is. The most critical issue the question already hints at is **maintainability**: the code is likely poorly structured, undocumented, and fragile. Any future changes or bug fixes will be extremely costly and error-prone. Beyond that, there are likely serious gaps in **security** (no proper authentication, data validation), **performance** (not tested under real loads), **reliability** (edge cases not handled), and **scalability**. A system that appears to work smoothly in a demo can fail badly under real-world conditions.

Third, you should propose a constructive path forward. Rather than simply refusing, you could offer to use the prototype as a **validated blueprint** and develop a proper system built on its design insights — this time with correct architecture, thorough testing, and documentation. This actually saves time overall

compared to the cost of fixing a failing production system later. You might estimate the additional cost and time required, showing the client that the investment now prevents far greater costs down the line.

Finally, document your concerns in writing. If after a thorough explanation the client still insists on deploying the prototype, you should formally document your professional objections and the risks you've communicated. This protects you legally and professionally, and sometimes the act of formal documentation makes clients reconsider. Your obligation as a software engineer is not just to deliver what the client asks for, but to act in their genuine best interest — and delivering a fragile prototype as a production system does not serve that interest.